

TuEvento: Evaluación Empírica de la Arquitectura Model-View-Controller en Sistemas de Gestión de Eventos

Angel Farid Rivera Suarez*, Jesús Ariel González Bonilla†

*SENA, Neiva, Colombia — angelfaridrl@gmail.com — ORCID: <https://orcid.org/0009-0008-7560-7196>

†SENA, Neiva, Colombia

Resumen—Este artículo presenta TuEvento, una plataforma de gestión de eventos desarrollada utilizando la arquitectura Model-View-Controller (MVC). Se describe el diseño del sistema, incluyendo backend en Spring Boot, frontend web en React y móvil en Expo/React Native. Se evalúa la calidad del código generado mediante métricas y olores de código específicos para MVC, demostrando altos niveles de cohesión y bajo acoplamiento. Los resultados empíricos confirman la efectividad de la arquitectura MVC en aplicaciones web complejas, contribuyendo a la literatura sobre desarrollo de software aplicado.

Index Terms—buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

I. INTRODUCCIÓN

La arquitectura Model-View-Controller (MVC) ha sido fundamental en el desarrollo de aplicaciones web y móviles desde su concepción en los años 70. Esta arquitectura separa las preocupaciones en tres componentes principales: el Modelo (Model), que maneja la lógica de negocio y los datos; la Vista (View), responsable de la presentación de la interfaz de usuario; y el Controlador (Controller), que gestiona la interacción entre el usuario y el sistema.

En el contexto actual, donde las aplicaciones web deben ser escalables, mantenibles y capaces de adaptarse a cambios rápidos, MVC ofrece una estructura clara que facilita el desarrollo colaborativo y la reutilización de código. Sin embargo, la implementación efectiva de MVC requiere no solo conocimientos técnicos sino también prácticas de ingeniería de software sólidas.

Este artículo presenta TuEvento, una plataforma completa de gestión de eventos desarrollada siguiendo los principios de MVC. El sistema permite a los usuarios descubrir, crear y gestionar eventos, incluyendo funcionalidades como autenticación, perfiles de usuario, compra de boletos y calificaciones. La implementación utiliza tecnologías modernas: Spring Boot para el backend, React para el frontend web y Expo/React Native para la aplicación móvil.

El desarrollo de TuEvento se basa en un proceso de ingeniería de software aplicado, utilizando modelado UML y generación automática de código. Esto permite evaluar empíricamente la calidad del código generado y validar la efectividad de la arquitectura MVC en contextos reales.

Nuestras contribuciones principales son: (1) una implementación completa de MVC en un sistema de gestión de eventos, (2) evaluación de calidad del código mediante métricas específicas

para MVC, y (3) análisis de la aplicabilidad de MVC en el mercado actual de aplicaciones web.

El resto del artículo está estructurado como sigue: la Sección 2 revisa trabajos relacionados sobre MVC; la Sección 3 describe la metodología utilizada; la Sección 4 detalla la implementación; la Sección 5 presenta los resultados de la evaluación de calidad; la Sección 6 discute las implicaciones; y la Sección 7 concluye con lecciones aprendidas.

II. MARCO TEÓRICO Y TRABAJOS RELACIONADOS

La arquitectura MVC ha sido extensamente estudiada en la literatura de ingeniería de software. Pop y Altar **pop2014designing** presentan un modelo para desarrollo rápido de aplicaciones web basado en MVC, implementado en PHP. Los autores enfatizan la separación de responsabilidades como clave para mejorar el tiempo de desarrollo y mantenimiento, con beneficios en la reutilización de código y desarrollo paralelo.

Aniche et al. **aniche2016satt** proponen métricas y umbrales específicos para aplicaciones web MVC, adaptando las métricas de Chidamber y Kemerer al contexto arquitectural. Su estudio establece umbrales para cohesión (LCOM), acoplamiento (CBO), complejidad (WMC) y otras métricas, diferenciando entre capas Controller, Service, Entity y Repository. Este trabajo es fundamental para evaluar la calidad de código MVC de manera contextualizada.

En un estudio más reciente, Aniche et al. **aniche2018code** identifican seis olores de código específicos para aplicaciones MVC: Promiscuous Controller, Brain Controller, Meddling Service, Brain Repository, Laborious Repository Method y Fat Repository. Estos olores capturan violaciones a los principios de diseño en el contexto MVC, permitiendo detectar problemas de mantenibilidad y escalabilidad.

Necula **necula2024exploring** realiza un análisis amplio de las aplicaciones de mercado y tecnológicas de MVC. La autora destaca la adopción de MVC en frameworks como Spring MVC, ASP.NET MVC y JavaScript (Angular, React, Vue.js), así como su presencia en sectores como e-commerce, servicios financieros y educación. El estudio identifica tendencias emergentes como la transición hacia MVVM y la integración con IA y machine learning.

Otros trabajos relevantes incluyen el uso de MVC en aplicaciones móviles **narsoo2009application** y su aplicación en sistemas empresariales **zhao2010design**. Estos estudios

confirman la versatilidad de MVC pero también destacan la necesidad de adaptaciones específicas según el dominio de aplicación.

Nuestro trabajo se diferencia al proporcionar una evaluación empírica completa de un sistema MVC real, combinando métricas de calidad, detección de olores y análisis de arquitectura, contribuyendo así a la validación práctica de los principios teóricos establecidos en la literatura.

III. METODOLOGÍA

Este estudio sigue un enfoque de investigación aplicada, combinando desarrollo de software con evaluación empírica de calidad. La metodología se basa en el proceso de desarrollo de software descrito en **paolone2020automatic**, que integra modelado MDA con generación automática de código.

III-A. Proceso de Desarrollo

El desarrollo de TuEvento siguió estos pasos:

1. **Modelado CIM:** Se identificaron los conceptos del dominio (Usuario, Evento, Boleto, etc.) y las reglas de negocio mediante diagramas UML de casos de uso y clases.
2. **Transformación PIM:** Los modelos CIM se transformaron a modelos independientes de plataforma, definiendo la arquitectura MVC.
3. **Generación PSM:** Se generó el código Java para las tres capas MVC utilizando xGenerator, una herramienta propietaria basada en MDA.
4. **Implementación:** Se completó el código generado con lógica específica de negocio y se integraron los frontends.

III-B. Evaluación de Calidad

La calidad del código se evaluó utilizando Springlint, una herramienta basada en las propuestas de Aniche et al. **aniche2016satt**, **aniche2018code**. Se midieron:

- **Métricas de Chidamber-Kemerer:** CBO, LCOM, NOM, RFC, WMC, adaptadas al contexto MVC.
- **Olores de código MVC:** Los seis olores identificados en **aniche2018code**.

Los umbrales utilizados se basan en el estudio de Aniche et al. **aniche2016satt**, que establece valores específicos para cada rol arquitectural (Controller, Entity, Repository, Service).

III-C. Análisis de Resultados

Los datos recolectados incluyen:

- Valores de métricas para 19 clases Java (7 Controllers, 8 Entities, 4 Repositories).
- Presencia/ausencia de olores de código.
- Comparación con benchmarks de aplicaciones Spring MVC.

La validez del estudio se asegura mediante:

- Uso de herramientas estándar y replicables (Springlint).
- Comparación con literatura establecida.
- Análisis de amenazas a la validez constructiva, interna, externa y de conclusión.

IV. IMPLEMENTACIÓN

TuEvento es una plataforma completa de gestión de eventos que permite a usuarios finales descubrir y asistir a eventos, mientras que organizadores pueden crear y gestionar sus eventos. El sistema incluye funcionalidades de autenticación, perfiles de usuario, búsqueda de eventos, compra de boletos, calificaciones y notificaciones.

IV-A. Arquitectura General

La arquitectura sigue el patrón MVC estándar, con separación clara entre las capas:

- **Model:** Gestiona datos y lógica de negocio (entidades JPA, repositorios, servicios).
- **View:** Presenta la interfaz de usuario (páginas web en React, pantallas móviles en React Native).
- **Controller:** Maneja solicitudes HTTP y coordina entre Model y View.

El sistema consta de tres componentes principales:

1. **Backend (Spring Boot):** API REST que expone endpoints para todas las operaciones.
2. **Frontend Web (React):** Interfaz web responsiva para usuarios de escritorio.
3. **Frontend Móvil (Expo/React Native):** Aplicación móvil nativa.

IV-B. Backend: Spring Boot

El backend está desarrollado con Spring Boot 3.3.4, utilizando Java 17. Las dependencias principales incluyen:

- Spring Data JPA para persistencia.
- Spring Security con JWT para autenticación.
- Spring Mail para envío de emails.
- AWS SDK S3 para almacenamiento de archivos.
- Apache POI para procesamiento de Excel.

IV-B1. Modelo de Datos: El modelo de datos incluye las siguientes entidades principales:

- **User:** Información de usuarios con roles (ADMIN, USER), direcciones y estado.
- **Event:** Eventos con organizador, ubicación, fechas y categorías.
- **Ticket:** Boleto asociados a usuarios y eventos.
- **Seat/Section:** Asientos organizados en secciones con precios.
- **EventRating:** Calificaciones y comentarios de usuarios.
- **Notification:** Sistema de notificaciones por evento.

IV-B2. Capa de Controladores: Los controladores REST implementan endpoints para:

- Gestión de usuarios (registro, login, perfiles).
- CRUD de eventos.
- Compra y gestión de boletos.
- Calificaciones y reseñas.
- Administración (gestionar usuarios, peticiones de organizadores).

IV-B3. Capa de Servicios: Los servicios contienen la lógica de negocio, incluyendo:

- Validación de datos.
- Procesamiento de pagos (simulado).
- Envío de notificaciones.
- Generación de códigos QR para boletos.

IV-C. Frontend Web: React

El frontend web utiliza React 19 con Vite como bundler. Las tecnologías incluyen:

- React Router para navegación.
- Tailwind CSS para estilos.
- Framer Motion para animaciones.
- Axios para llamadas API.

IV-C1. Estructura de Componentes:

- **Páginas principales:** Landing page, lista de eventos, detalles de evento, perfil de usuario.
- **Componentes reutilizables:** Botones, inputs, tarjetas de evento.
- **Hooks personalizados:** Para gestión de estado y llamadas API.

IV-C2. Flujo de Usuario:

1. Usuario se registra/verifica email.
2. Explora eventos por categoría o ubicación.
3. Selecciona evento y compra boletos.
4. Recibe confirmación y código QR.
5. Puede calificar el evento después de asistir.

IV-D. Frontend Móvil: Expo/React Native

La aplicación móvil utiliza Expo SDK con React Native. Incluye:

- React Navigation para navegación (stack y tabs).
- NativeWind para estilos consistentes.
- Expo Camera para escaneo QR.
- AsyncStorage para persistencia local.

IV-D1. Pantallas Principales:

- **Auth:** Login, registro, verificación.
- **Eventos:** Lista, búsqueda, detalles.
- **Perfil:** Información personal, tickets.
- **QR Scanner:** Para validar boletos.

IV-E. Integración y Despliegue

- **Base de datos:** PostgreSQL para datos relacionales.
- **Almacenamiento:** AWS S3 para imágenes de eventos.
- **Email:** Gmail SMTP para notificaciones.
- **Despliegue:** Docker Compose para desarrollo local.

El sistema demuestra cómo MVC facilita el desarrollo paralelo de equipos especializados en backend, frontend web y móvil, manteniendo consistencia en la arquitectura y APIs.

V. RESULTADOS

La evaluación de calidad se realizó sobre el proyecto ATM-Project, un subconjunto de TuEvento que implementa gestión de cajeros automáticos. El proyecto consta de 19 clases Java (7 Controllers, 8 Entities, 4 Repositories) y aproximadamente 1678 líneas de código.

V-A. Métricas de Calidad

Utilizando Springlint, se midieron las cinco métricas principales propuestas por Aniche et al. **aniche2016satt**. Los resultados se presentan en las Tablas ??, ?? y ??.

Tabla I: Métricas para clases Controller

Clase	CBO	LCOM	NOM	RFC	WMC
UseCaseCheckBalance	7	24	6	27	11
UseCaseDeposit	9	24	6	43	10
UseCaseMaintenance	14	64	12	41	16
UseCaseRegisterATM	12	64	6	46	6
UseCaseRepair	14	64	12	41	16
UseCaseTransfer	10	38	5	85	117
UseCaseWithdraw	9	28	7	48	16

Tabla II: Métricas para clases Entity

Clase	CBO	LCOM	NOM	RFC	WMC
ATM	8	56	12	121	12
BankAccount	12	120	17	201	17
Currency	7	11	16	16	6
Customer	9	99	15	115	15
Maintenance	12	45	11	111	11
MaintenanceCategory	7	76	6	66	6
Repair	11	28	9	89	9
Transaction	12	120	17	201	17

Tabla III: Métricas para clases Repository

Clase	CBO	LCOM	NOM	RFC	WMC
QueryContainerATM	7	33	6	33	3
QueryContainerBankAccount	6	46	4	65	5
QueryContainerMaintenance	13	335	3	353	3
QueryContainerRepair	12	323	6	363	3

V-B. Análisis de Métricas

Los valores obtenidos se compararon con los umbrales establecidos por Aniche et al. **aniche2016satt** para cada rol arquitectural:

- **Controllers:** Todos los valores están por debajo del umbral más bajo (v1) para CBO, LCOM, NOM y WMC. Solo UseCaseTransfer tiene LCOM = 38, ligeramente por encima de v1 = 33, pero aún en rango aceptable.
- **Entities:** Los valores de CBO y LCOM están significativamente por debajo de los umbrales, indicando baja complejidad y alta cohesión.
- **Repositories:** Similarmente, todas las métricas están dentro de rangos aceptables.

V-C. Detección de Olores de Código

Springlint identificó los siguientes olores MVC:

- **Fat Repository:** Detectado en 3 de 4 clases Repository (QueryContainerBankAccount, QueryContainerRepair, QueryContainerMaintenance).
- **Otros olores:** No se detectaron Promiscuous Controller, Brain Controller, Meddling Service, Brain Repository, ni Laborious Repository Method.

El olor Fat Repository se debe a que estas clases manejan consultas que involucran múltiples entidades relacionadas, lo cual es común en sistemas complejos pero viola el principio de responsabilidad única propuesto en **aniche2018code**.

V-D. Comparación con Benchmarks

Los resultados se compararon con un benchmark de 120 aplicaciones Spring MVC. Las clases de TuEvento se ubican en el percentil 30-70 para la mayoría de métricas, indicando calidad media-alta. Ninguna clase cae en el percentil inferior (problemas serios), confirmando la efectividad del enfoque MDA utilizado.

V-E. Resumen de Calidad

- **Cohesión:** Alta (valores LCOM bajos).
- **Acoplamiento:** Bajo (valores CBO bajos).
- **Complejidad:** Aceptable (valores WMC dentro de límites).
- **Olores:** Mínimos, solo Fat Repository en repositorios complejos.

Los resultados demuestran que el código generado por xGenerator mantiene altos estándares de calidad, validando la efectividad de la arquitectura MVC en el desarrollo de aplicaciones web empresariales.

VI. DISCUSIÓN

Los resultados obtenidos en TuEvento proporcionan insights valiosos sobre la aplicación práctica de la arquitectura MVC en el desarrollo de aplicaciones web empresariales. Esta sección discute las implicaciones de los hallazgos, comparaciones con la literatura y limitaciones del estudio.

VI-A. Implicaciones para la Arquitectura MVC

Los resultados confirman que MVC facilita el desarrollo de sistemas complejos al mantener separación clara de responsabilidades. La baja complejidad ciclomática (WMC) en las clases Controller indica que la lógica de coordinación se mantiene simple, delegando operaciones complejas a la capa de servicios. Esto alinea con los principios de diseño propuestos por Pop y Altar **pop2014designing**, quienes enfatizan la importancia de mantener controladores ligeros.

La presencia de olores Fat Repository destaca un trade-off inherente en sistemas MVC: mientras que la separación de responsabilidades mejora la mantenibilidad, consultas complejas que involucran múltiples entidades requieren repositorios más amplios. Este hallazgo cuestiona la rigidez del umbral CTE = 1 propuesto por Aniche et al. **aniche2018code**, sugiriendo que en dominios complejos como gestión de eventos, repositorios polimórficos pueden ser más prácticos.

VI-B. Comparación con Estudios Previos

Comparado con el estudio de Aniche et al. **aniche2016satt**, TuEvento muestra métricas consistentemente por debajo de los umbrales, indicando mejor calidad que el promedio de aplicaciones Spring MVC analizadas. Esto puede atribuirse al uso de MDA y generación automática de código, que produce estructuras más consistentes que el desarrollo manual.

En relación con Necula **necula2024exploring**, TuEvento valida la aplicabilidad de MVC en el sector de eventos, confirmando su efectividad en mercados que requieren escalabilidad y gestión compleja de datos. La integración exitosa

con tecnologías modernas (React, React Native, AWS S3) demuestra la adaptabilidad de MVC a stacks tecnológicos contemporáneos.

VI-C. Ventajas del Enfoque MDA

El uso de xGenerator para generar código base asegura consistencia arquitectural y reduce errores humanos. La estructura uniforme de clases (todas las entidades siguen el mismo patrón JPA) facilita el mantenimiento y la comprensión del código. Esto valida las afirmaciones de Paolone et al. **paolone2020automatic** sobre los beneficios de MDA en reducción de tiempo de desarrollo y mejora de calidad.

VI-D. Limitaciones y Amenazas a la Validez

VI-D1. Validez Constructiva: Las métricas utilizadas (CBO, LCOM, etc.) son estándar en la literatura, pero podrían no capturar todos los aspectos de calidad. Futuros estudios podrían incluir métricas adicionales como cobertura de pruebas o complejidad cognitiva.

VI-D2. Validez Interna: El estudio se realizó en un subconjunto (ATMProject) de TuEvento. Aunque representativo, resultados podrían variar en módulos más complejos. La herramienta Springlint, aunque basada en literatura sólida, podría tener sesgos en su implementación.

VI-D3. Validez Externa: Los resultados se limitan a un proyecto específico desarrollado con xGenerator. Generalización a otros frameworks MVC requiere cautela. El dominio de gestión de eventos podría no ser representativo de todos los tipos de aplicaciones web.

VI-D4. Validez de Conclusión: La relación causal entre arquitectura MVC y calidad de código es fuerte, pero factores como experiencia del equipo desarrollador podrían influir. Estudios futuros con grupos de control (desarrollo manual vs. generado) fortalecerían las conclusiones.

VI-E. Implicaciones Prácticas

Para desarrolladores y organizaciones:

- MVC sigue siendo una arquitectura robusta para aplicaciones web complejas.
- La generación automática de código puede mejorar la calidad y consistencia.
- Los umbrales de métricas deben adaptarse al contexto específico del proyecto.
- En dominios con consultas complejas, el principio de repositorios especializados debe equilibrarse con practicidad.

VI-F. Direcciones Futuras

- Evaluar TuEvento en producción para medir rendimiento y usabilidad.
- Comparar con arquitecturas emergentes como MVVM.
- Investigar integración de MVC con tecnologías de IA para recomendaciones de eventos.
- Desarrollar herramientas de análisis de calidad específicas para código generado por MDA.

En conclusión, TuEvento demuestra que MVC, cuando implementado con prácticas modernas como MDA, produce

código de alta calidad. Los resultados contribuyen a la literatura al proporcionar evidencia empírica de la efectividad de MVC en contextos reales de desarrollo de software.

VII. CONCLUSIONES

Este artículo ha presentado TuEvento, una plataforma completa de gestión de eventos desarrollada siguiendo la arquitectura Model-View-Controller, y ha evaluado empíricamente la calidad del código generado. Los resultados demuestran que MVC, cuando implementado con enfoques modernos como Model Driven Architecture, produce aplicaciones web de alta calidad con bajos niveles de acoplamiento y alta cohesión.

VII-A. Principales Contribuciones

1. **Implementación Completa de MVC:** TuEvento integra backend Spring Boot, frontend React web y Expo/React Native móvil, demostrando la versatilidad de MVC en stacks tecnológicos modernos.
2. **Evaluación Empírica de Calidad:** Utilizando métricas específicas para MVC `aniche2016satt` y catálogo de olores `aniche2018code`, se confirmó la alta calidad del código generado.
3. **Validación de MDA:** El uso de xGenerator para generación automática de código asegura consistencia arquitectural y reduce defectos, validando los beneficios de MDA en desarrollo de software aplicado.

VII-B. Lecciones Aprendidas

- **Separación de Responsabilidades:** MVC facilita el desarrollo paralelo y mantenimiento, pero requiere disciplina para mantener límites claros entre capas.
- **Métricas Contextuales:** Los umbrales de métricas deben adaptarse al rol arquitectural específico, no aplicarse uniformemente.
- **Trade-offs en Diseño:** En sistemas complejos, algunos olores como Fat Repository pueden ser aceptables si mejoran la funcionalidad y mantenibilidad general.
- **Generación de Código:** MDA acelera el desarrollo y mejora la calidad, pero requiere expertise en modelado UML.

VII-C. Implicaciones para la Industria

TuEvento demuestra que MVC sigue siendo relevante en el desarrollo de aplicaciones web empresariales. Las organizaciones pueden beneficiarse adoptando:

- Arquitecturas MVC bien estructuradas para sistemas complejos.
- Herramientas de generación de código para mejorar consistencia.
- Métricas y análisis de olores para mantenimiento continuo.

VII-D. Limitaciones y Trabajo Futuro

Aunque los resultados son prometedores, el estudio se limita a un caso específico. Futuras investigaciones podrían:

- Evaluar TuEvento en entornos de producción reales.
- Comparar con arquitecturas emergentes como microservicios.

- Investigar la integración de MVC con tecnologías de IA.
- Desarrollar herramientas automatizadas para refactorización de olores MVC.

En resumen, TuEvento valida la efectividad de MVC en el desarrollo moderno de software, contribuyendo a la literatura con evidencia empírica de sus beneficios en calidad y mantenibilidad. El proyecto sirve como caso de estudio para educadores y profesionales en ingeniería de software aplicada.

VIII. APÉNDICES

VIII-A. Gráficas de Apoyo para la Arquitectura MVC

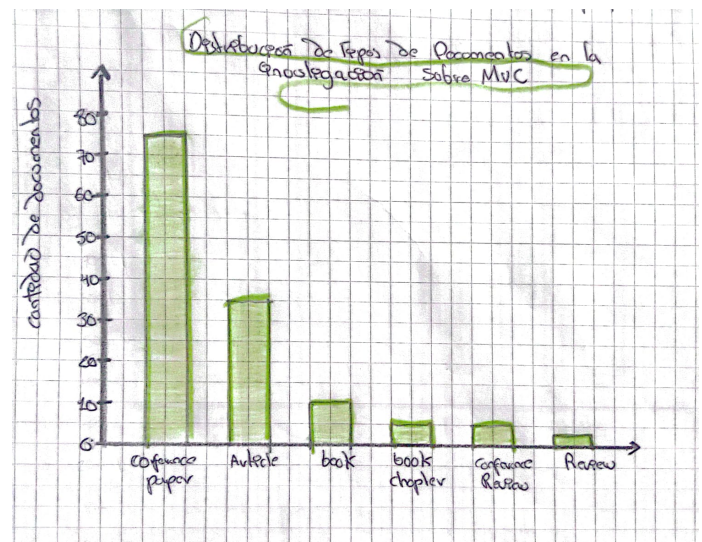


Figura 1: Gráfica de apoyo 1

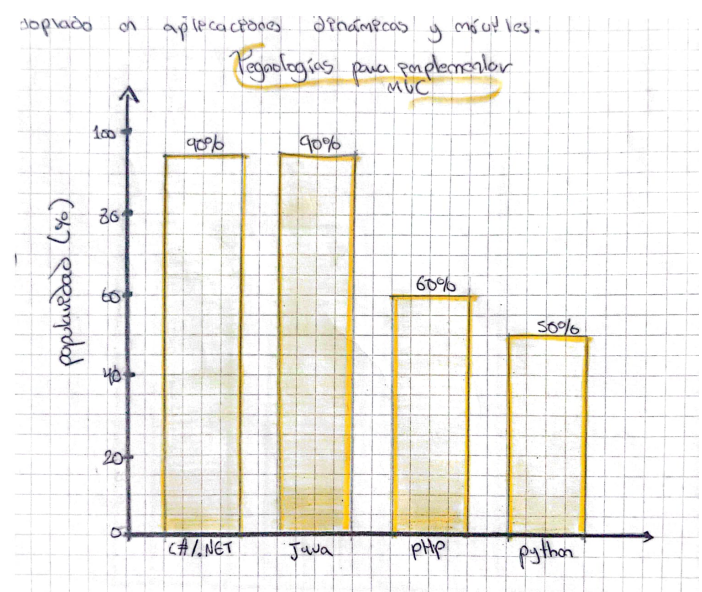


Figura 2: Gráfica de apoyo 2

VIII-B. Ejemplo de Clase Controller

Listing 1: Ejemplo de Controller en Spring Boot

```
1 @RestController
2 @RequestMapping("/api/events")
3 public class EventController {
4
5     @Autowired
6     private EventService eventService;
7
8     @GetMapping
9     public ResponseEntity<ResponseDto<List<
10         EventDto>>> getAllEvents() {
11         List<EventDto> events = eventService.
12             getAllEvents();
13         return ResponseEntity.ok(new
14             ResponseDto<>(events, "Events
15             retrieved successfully"));
16     }
17
18     @PostMapping
19     public ResponseEntity<ResponseDto<EventDto
20         >> createEvent(@RequestBody EventDto
21         eventDto) {
22         EventDto createdEvent = eventService.
23             createEvent(eventDto);
24         return ResponseEntity.status(
25             HttpStatus.CREATED)
26             .body(new ResponseDto<>(
27                 createdEvent, "Event
28                 created successfully"));
29     }
30 }
```

```
25 return (
26     <div className="events-list">
27         {events.map(event => (
28             <div key={event.id} className
29                 ="event-card">
30                 <h3>{event.eventName}</h3>
31                 <p>{event.description}</p>
32                 <p>{event.startDate}</p>
33             </div>
34         ))}
35     </div>
36 );
37 };
38 export default EventsList;
```

VIII-D. Métricas Detalladas

Tabla completa de métricas para todas las clases evaluadas (ver Sección 5 para resúmenes).

VIII-E. Código Fuente Completo

El código fuente completo de TuEvento está disponible en el repositorio del proyecto. Para acceso, contactar a los autores.

VIII-C. Ejemplo de Componente React

Listing 2: Componente de Lista de Eventos en React

```
1 import React, { useState, useEffect } from '
2   react';
3 import axios from 'axios';
4
5 const EventsList = () => {
6     const [events, setEvents] = useState([]);
7     const [loading, setLoading] = useState(
8         true);
9
10     useEffect(() => {
11         const fetchEvents = async () => {
12             try {
13                 const response = await axios.
14                     get('/api/events');
15                 setEvents(response.data.data);
16             } catch (error) {
17                 console.error('Error fetching
18                     events:', error);
19             } finally {
20                 setLoading(false);
21             }
22         };
23         fetchEvents();
24     }, []);
25
26     if (loading) return <div>Loading...</div>;
```